

```

import numpy as np
import matplotlib.pyplot as plt

def il_v2(r):
    r = np.asarray(r, dtype=float)
    return 2 * np.sqrt(r) / (1 + r) - 1

# Sanity checks
print(il_v2(1))    # should be 0
print(il_v2(2))    # about -5.72%
print(il_v2(0.5))  # same as r=2, IL is symmetric in log terms

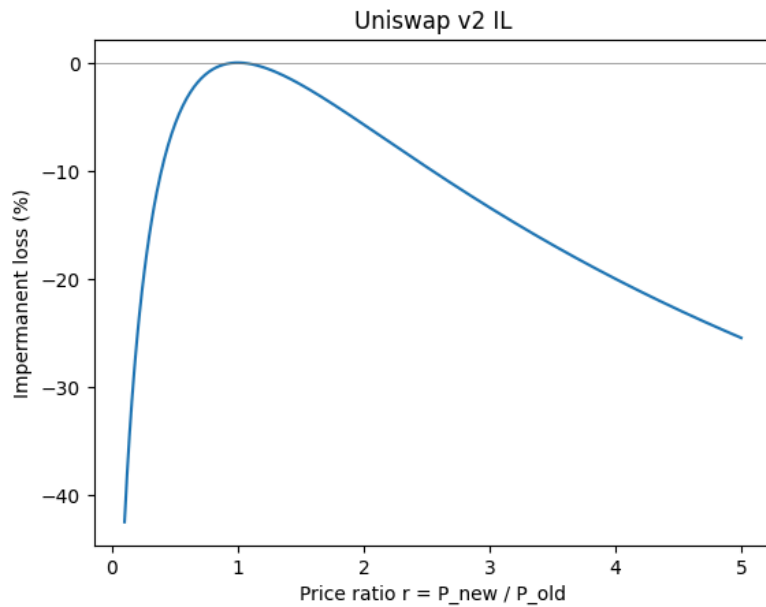
# Plot IL vs price ratio
r = np.linspace(0.1, 5, 500)
plt.plot(r, il_v2(r) * 100)
plt.axhline(0, color="gray", lw=0.5)
plt.xlabel("Price ratio r = P_new / P_old")
plt.ylabel("Impermanent loss (%)")
plt.title("Uniswap v2 IL")
plt.show()

```

```

0.0
-0.05719095841793653
-0.05719095841793653

```



```

import numpy as np
import matplotlib.pyplot as plt

def v3_value_per_L(p, pa, pb):
    p = np.asarray(p, dtype=float)
    sp = np.sqrt(np.clip(p, pa, pb))
    x = 1 / sp - 1 / np.sqrt(pb)  # base token per unit L
    y = sp - np.sqrt(pa)          # quote token per unit L
    return x * p + y

def il_v3(p0, p1, pa, pb):
    p1 = np.asarray(p1, dtype=float)
    L = 1.0 / v3_value_per_L(p0, pa, pb)  # normalise start value to 1
    lp = L * v3_value_per_L(p1, pa, pb)
    sp0 = np.sqrt(p0)
    x0 = L * (1 / sp0 - 1 / np.sqrt(pb))
    y0 = L * (sp0 - np.sqrt(pa))
    hodl = x0 * p1 + y0
    return lp / hodl - 1

p0 = 1.0
p1 = np.linspace(0.3, 3, 600)

# Plot v2 vs v3

```

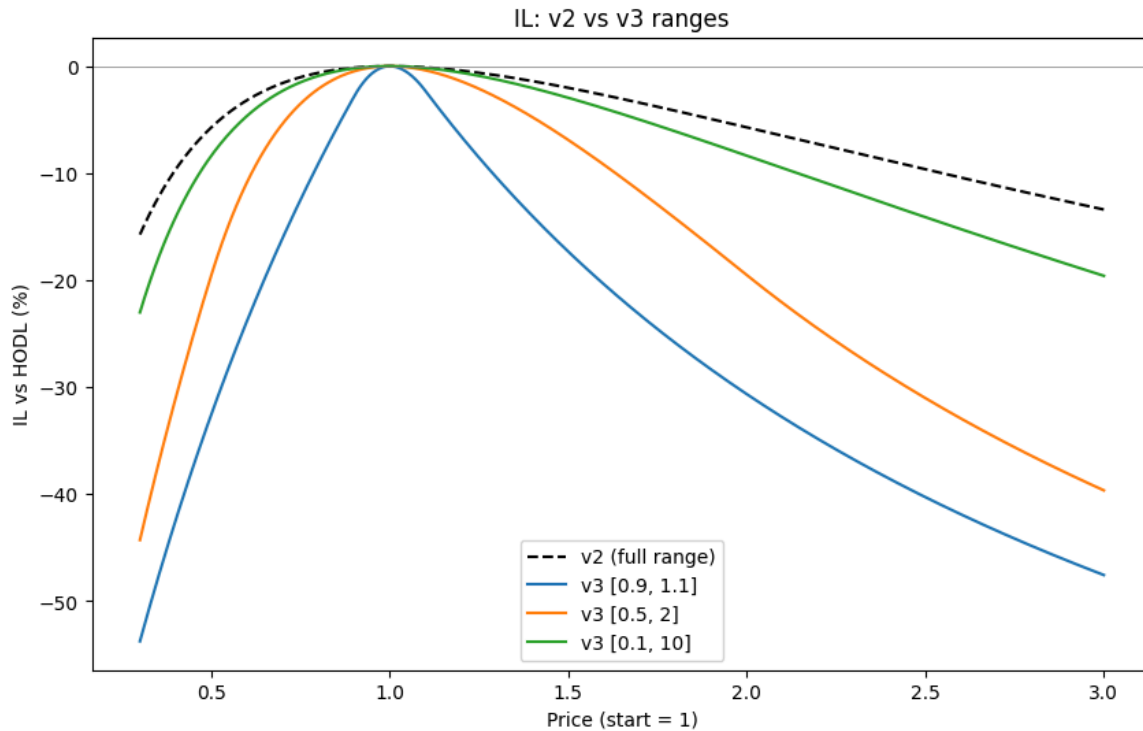
```

plt.figure(figsize=(10, 6))
plt.plot(p1, il_v2(p1 / p0) * 100, "k--", label="v2 (full range)")
for pa, pb in [(0.9, 1.1), (0.5, 2), (0.1, 10)]:
    plt.plot(p1, il_v3(p0, p1, pa, pb) * 100, label=f"v3 [{pa}, {pb}]")

plt.axhline(0, color="gray", lw=0.5)
plt.xlabel("Price (start = 1)")
plt.ylabel("IL vs HODL (%)")
plt.legend()
plt.title("IL: v2 vs v3 ranges")
plt.show()

print(f"IL v3 at start price: {il_v3(1, 1, 0.9, 1.1)}") # should be 0

```



IL v3 at start price: 0.0

Fees vs IL

```

def liquidity_multiplier(p0, pa, pb):
    L3 = 1.0 / v3_value_per_L(p0, pa, pb)
    L2 = 1.0 / (2 * np.sqrt(p0))
    return L3 / L2

def net_return(p0, p1, pa, pb, fee_v2):
    """Net return vs HODL: IL + fees. fee_v2 = fee return a v2 LP earns over the period."""
    p1 = np.asarray(p1, dtype=float)
    m = liquidity_multiplier(p0, pa, pb)
    in_range = (p1 >= pa) & (p1 <= pb)
    v3 = il_v3(p0, p1, pa, pb) + np.where(in_range, fee_v2 * m, 0.0)
    v2 = il_v2(p1 / p0) + fee_v2
    return v2, v3

p0, fee_v2 = 1.0, 0.03 # assume 3% fee return for v2 over the period
p1 = np.linspace(0.3, 3, 600)

for pa, pb in [(0.9, 1.1), (0.5, 2), (0.1, 10)]:
    print(f"[{pa}, {pb}] multiplier = {liquidity_multiplier(p0, pa, pb):.1f}x")

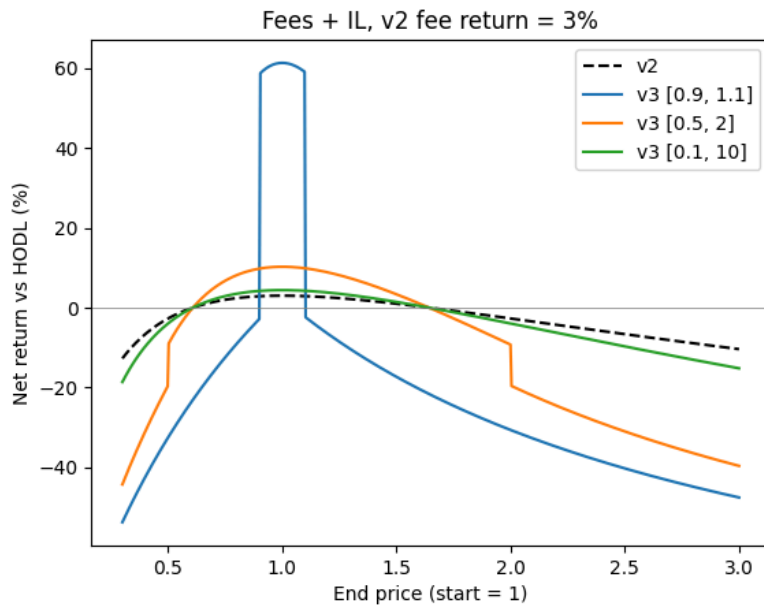
v2, _ = net_return(p0, p1, 0.9, 1.1, fee_v2)
plt.plot(p1, v2 * 100, "k--", label="v2")
for pa, pb in [(0.9, 1.1), (0.5, 2), (0.1, 10)]:
    _, v3 = net_return(p0, p1, pa, pb, fee_v2)
    plt.plot(p1, v3 * 100, label=f"v3 [{pa}, {pb}]")

plt.axhline(0, color="gray", lw=0.5)

```

```
plt.xlabel("End price (start = 1)")
plt.ylabel("Net return vs HODL (%)")
plt.legend()
plt.title(f"Fees + IL, v2 fee return = {fee_v2:.0%}")
plt.show()
```

```
[0.9, 1.1] multiplier = 20.4x
[0.5, 2] multiplier = 3.4x
[0.1, 10] multiplier = 1.5x
```



Real data: TVL and fee APY

```
import requests, pandas as pd
pools = requests.get("https://yields.llama.fi/pools").json()["data"]
c = [p for p in pools if p["project"] == "uniswap-v3" and p["chain"] == "Ethereum" and set(p["symbol"].split("-"))
for p in c: print(p["pool"], p["poolMeta"], p["tv1Usd"])]
```

```
665dc8bc-c79d-4800-97f7-304bf368e547 0.05% 95757782
49717ee2-9808-4288-b76d-e658195b7979 0.3% 44474601
8b3ed515-5e6f-449a-9b64-25113cda7a29 0.01% 8614730
61d9b656-e4e8-4b6b-8659-06131995b499 1% 828199
```

```
pid = [p["pool"] for p in c if p["poolMeta"] == "0.05%"][0]
h = requests.get("https://yields.llama.fi/chart/" + pid).json()["data"]
tv1 = pd.DataFrame(h)
tv1["day"] = pd.to_datetime(tv1["timestamp"]).dt.date
tv1 = tv1[["day", "tv1Usd"]].tail(90)
tv1.head()
```

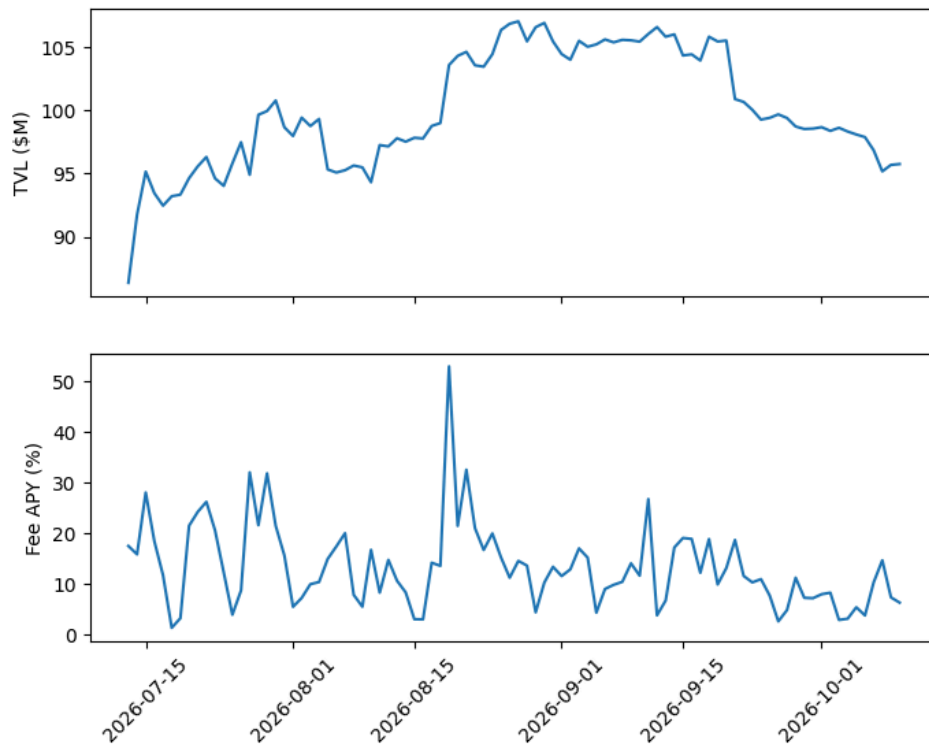
	day	tv1Usd	
1541	2026-07-13	86350961	
1542	2026-07-14	91739441	
1543	2026-07-15	95158089	
1544	2026-07-16	93444616	
1545	2026-07-17	92459624	

```
import requests, pandas as pd
import matplotlib.pyplot as plt
pid = "665dc8bc-c79d-4800-97f7-304bf368e547"
h = requests.get("https://yields.llama.fi/chart/" + pid).json()["data"]
df = pd.DataFrame(h).tail(90).copy()
df["day"] = pd.to_datetime(df["timestamp"]).dt.date
df = df[["day", "tv1Usd", "apyBase"]]
print(df.isna().sum())
print(df.describe())
```

```
fig, ax = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
ax[0].plot(df["day"], df["tv1Usd"] / 1e6)
ax[0].set_ylabel("TVL ($M)")
ax[1].plot(df["day"], df["apyBase"])
ax[1].set_ylabel("Fee APY (%)")
plt.xticks(rotation=45)
plt.show()
```

```
day      0
tv1Usd   0
apyBase  0
dtype: int64
```

	tv1Usd	apyBase
count	9.000000e+01	90.000000
mean	1.000199e+08	13.372043
std	4.605500e+06	8.260562
min	8.635096e+07	1.391440
25%	9.645126e+07	7.801540
50%	9.914261e+07	11.716655
75%	1.046031e+08	17.397330
max	1.070628e+08	52.970230



```
import numpy as np
end = int(pd.Timestamp.now().timestamp())
url = "https://coins.llama.fi/chart/coingecko:ethereum?end=" + str(end) + "&span=90&period=1d"
pr = requests.get(url).json()["coins"]["coingecko:ethereum"]["prices"]
px = pd.DataFrame(pr)
px["day"] = pd.to_datetime(px["timestamp"], unit="s").dt.date
px = px.groupby("day")["price"].last().reset_index()
m = df.merge(px, on="day")
print(len(m))
print(m[["day", "price"]].iloc[[0, -1]])
```

```
90
```

	day	price
0	2026-07-13	1776.392583
89	2026-10-10	2491.950472

```
p0 = m["price"].iloc[0]
price = m["price"].values
fdays = m["apyBase"].values / 100 / 365
v2_il = float(il_v2(price[-1] / p0))
v2_fees = fdays.sum()
print("v2 IL", v2_il, "fees", v2_fees, "net", v2_il + v2_fees)
res = pd.DataFrame({"width": [0.05, 0.2, 0.5]})
res["pa"] = p0 * (1 - res["width"])
```

```

res["pb"] = p0 * (1 + res["width"])
pairs = list(zip(res["pa"], res["pb"]))
res["mult"] = [float(liquidity_multiplier(p0, a, b)) for a, b in pairs]
res["in_range"] = [((price >= a) & (price <= b)).mean() for a, b in pairs]
res["fees"] = [(fday * ((price >= a) & (price <= b))).sum() * k for (a, b), k in zip(pairs, res["mult"])]
res["il"] = [float(il_v3(p0, price[-1], a, b)) for a, b in pairs]
res["net"] = res["il"] + res["fees"]
print(res[["width", "mult", "in_range", "fees", "il", "net"]])

```

```

v2 IL -0.014152232724499836 fees 0.032972160821917805 net 0.01881992809741797
width mult in_range fees il net
0 0.05 40.469042 0.088889 0.095280 -0.154117 -0.058837
1 0.20 10.378726 0.422222 0.165466 -0.117564 0.047902
2 0.50 4.198182 0.855556 0.128388 -0.061792 0.066596

```

```

ws = np.arange(0.02, 1.01, 0.02)
nets = []
for w in ws:
    a, b = p0 * (1 - w), p0 * (1 + w)
    k = float(liquidity_multiplier(p0, a, b))
    inr = (price >= a) & (price <= b)
    f = (fday * inr).sum() * k
    nets.append((float(il_v3(p0, price[-1], a, b)) + f) * 100)

plt.plot(ws * 100, nets, label="v3 net")
plt.axhline((v2_il + v2_fees) * 100, color="k", ls="--", label="v2 net")
plt.xlabel("Range half-width (%)")
plt.ylabel("Net return vs HODL (%)")
plt.legend()
plt.savefig("range_sweep.png", dpi=150)
plt.show()

```

